

draw.io

<https://plantumlonlineeditor.com/>

1. Program using class, object, package, overloaded methods, command line arguments

Folder structure:

```
src/  
└── com/  
    └── scsvmv/  
        └── javalab/  
            ├── Hello.java  
            ├── TestHello.java  
            └── hello/  
                └── Hello.java
```

Files:

- com/scsvmv/javalab/Hello.java
→ Simple Hello class (always prints *Hello World*)
- com/scsvmv/javalab/TestHello.java
→ Main class with command-line arguments
- com/scsvmv/javalab/hello/Hello.java
→ Overloaded wish() methods and Date usage

Steps to Compile the Program

1. Open Command Prompt / Terminal
2. Navigate to the src folder using cd command
Note: Always compile from the **src folder**, not inside package folders.
3. Compile all Java files

```
.....src>javac com/scsvmv/javalab/hello/Hello.java
```

```
.....src>javac com/scsvmv/javalab/Hello.java
```

```
.....src>javac com/scsvmv/javalab/TestHello.java
```

✓ .class files will be generated in the same package folders.

Steps to Run the Program

Run with command-line argument:

```
java com.scsvmv.javalab.TestHello Seetharaman
```

If no argument is passed:

```
java com.scsvmv.javalab.TestHello
```

Sample Output (with argument)

Hello World
Hello Seetharaman
Hello Seetharaman | Date: Thu Feb 05 10:30:15 IST 2026

Output (without argument)

Hello World

Usage:

java com.scsvmv.javalab.TestHello <name>

Example:

java com.scsvmv.javalab.TestHello Seetharaman

TestHello.java

```
package com.scsvmv.javalab;

import com.scsvmv.javalab.hello.Hello;

/*
 * TestHello class
 * Demonstrates passing command-line arguments
 */
public class TestHello {

    public static void main(String[] args) {

        // Using Hello class from parent package
        Hello simpleHello = new Hello();
        simpleHello.wish();

        // Using Hello class from child package
        com.scsvmv.javalab.hello.Hello h =
            new com.scsvmv.javalab.hello.Hello();

        // Check for command-line argument
        if (args.length > 0) {
            h.wish(args[0]);          // pass CLI argument
            h.wishWithDate(args[0]);
        } else {
            System.out.println("Usage:");
            System.out.println("java com.scsvmv.javalab.TestHello <name>");
            System.out.println("Example:");
            System.out.println("java com.scsvmv.javalab.TestHello Student");
        }
    }
}
```

Hello.java (com.scsvmv.javalab)

```
package com.scsvmv.javalab;
```

```
/*
 * Simple Hello class
 * Always wishes Hello World
 */
public class Hello {

    public void wish() {
        System.out.println("Hello World");
    }
}
```

Hello.java (com.scsvmv.javalab.hello)

```
package com.scsvmv.javalab.hello;
```

```
import java.util.Date;
```

```
/*
 * Hello class
 * Demonstrates method overloading and command-line arguments
 */
public class Hello {

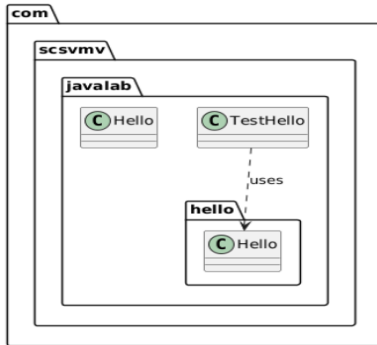
    // Default constructor
    public Hello() {
    }

    // Generic wish
    public void wish() {
        System.out.println("Hello World");
    }

    // Overloaded wish method with name parameter
    public void wish(String name) {
        System.out.println("Hello " + name);
    }

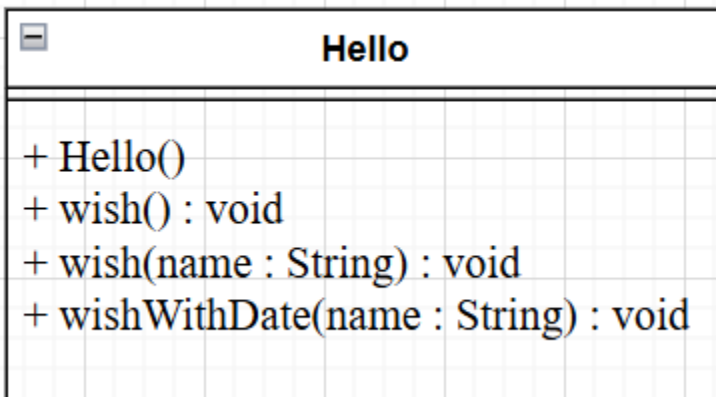
    // Wish with name and today's date
    public void wishWithDate(String name) {
        Date today = new Date();
        System.out.println("Hello " + name + " | Date: " + today);
    }
}
```

Package Diagram

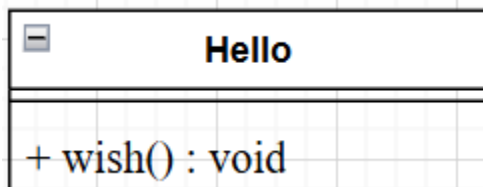


Class Diagrams

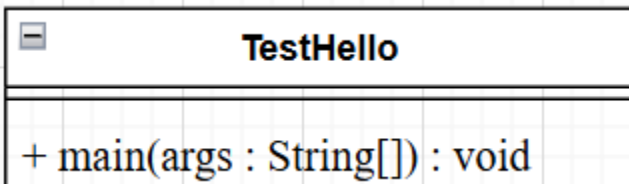
com.scsvmv.javalab.hello.Hello



com.scsvmv.javalab.Hello



com.scsvmv.javalab.TestHello



Viva Questions: Basic Level

1. What is a package in Java? Why do we use packages in your program?

2. What is the role of the Hello class in com.scsvmv.javalab and in other package ?
3. How does the TestHello class instantiate objects of Hello?
4. What are overloaded methods? Which methods are overloaded in your program?
5. How do you pass command-line arguments to the wish method?
6. What is the difference between Hello in the parent package and Hello in the child package?

Viva Questions: Intermediate Level

1. How does the wishWithDate() method work? Which Java class does it use?
2. What happens if no command-line argument is provided in TestHello?
3. How do you compile and run your program considering the package structure?
4. Can com.scsvmv.javalab.hello.Hello access private members of com.scsvmv.javalab.Hello? Why or why not?

Viva Questions: Advanced Level

1. How would you modify the program to print the current date and time in a specific format (like dd-MM-yyyy HH:mm:ss)?
2. If multiple developers are working on this program, how does package separation help in project organization?
3. Explain the difference between overloaded constructor vs overloaded method in terms of design and usability in your program.
4. How would you convert this program into a jar and execute it from the command line?
5. Discuss how UML diagrams can help in understanding and maintaining this program.

Extra Practical / Trick Questions

1. What happens if you write Hello h = new Hello(); in TestHello without importing the child package class?
2. Can you have two classes with the same name in the same package? Why or why not?
3. How do you handle multiple overloaded methods with the same parameters but different return types?
4. Why is it important to pass command-line arguments to wish() instead of hardcoding the name?

Prog2: Program to explore java.lang.Math class methods and command line arguments, custom package

project-folder/

```

├── Enter2Float.java
└── operations/
    └── BasicOperations.java

```

operations/BasicOperations.java

```
package operations;
```

```
public class BasicOperations {
```

```

private float num1;
private float num2;

// Constructor for float values
public BasicOperations(float num1, float num2) {
    this.num1 = num1;
    this.num2 = num2;
}

// Overloaded constructor for int values
public BasicOperations(int num1, int num2) {
    // Convert ints to float
    this.num1 = (float) num1;
    this.num2 = (float) num2;
}

public void performOperations() {

    System.out.println("--- Basic Arithmetic ---");
    System.out.println("Addition: " + (num1 + num2));
    System.out.println("Subtraction: " + (num1 - num2));
    System.out.println("Multiplication: " + (num1 * num2));

    if (num2 != 0) {
        System.out.println("Division: " + (num1 / num2));
        System.out.println("Modulus: " + (num1 % num2));
    } else {
        System.out.println("Division and Modulus: Cannot divide by zero");
    }

    System.out.println("\n--- Math Class Methods ---");
    System.out.println("Absolute value of num1: " + Math.abs(num1));
    System.out.println("Maximum: " + Math.max(num1, num2));
    System.out.println("Minimum: " + Math.min(num1, num2));
    System.out.println("Power (num1^num2): " + Math.pow(num1, num2));
    System.out.println("Square root of num1: " + Math.sqrt(num1));
    System.out.println("Rounded num1: " + Math.round(num1));
    System.out.println("Floor of num1: " + Math.floor(num1));
    System.out.println("Ceil of num1: " + Math.ceil(num1));
    System.out.println("Random number (0-1): " + Math.random());
}
}

```

Enter2Float.java

```

import operations.BasicOperations;

```

```

public class Enter2Float {

    public static void main(String[] args) {

        if (args.length != 2) {
            System.out.println("Usage: java Enter2Float <num1> <num2>");
            return;
        }

        try {
            float num1 = Float.parseFloat(args[0]);
            float num2 = Float.parseFloat(args[1]);

            // Using float constructor
            BasicOperations obj1 = new BasicOperations(num1, num2);
            obj1.performOperations();

            // Using int constructor
            BasicOperations obj2 = new BasicOperations(10, 3);
            obj2.performOperations();

        } catch (NumberFormatException e) {
            System.out.println("Please enter valid float numbers.");
        }
    }
}

```

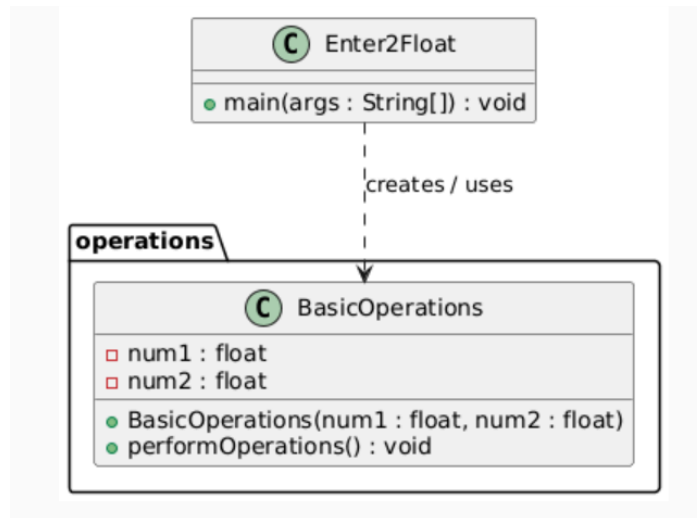
Compile & Run

Open CMD inside project-folder and run:

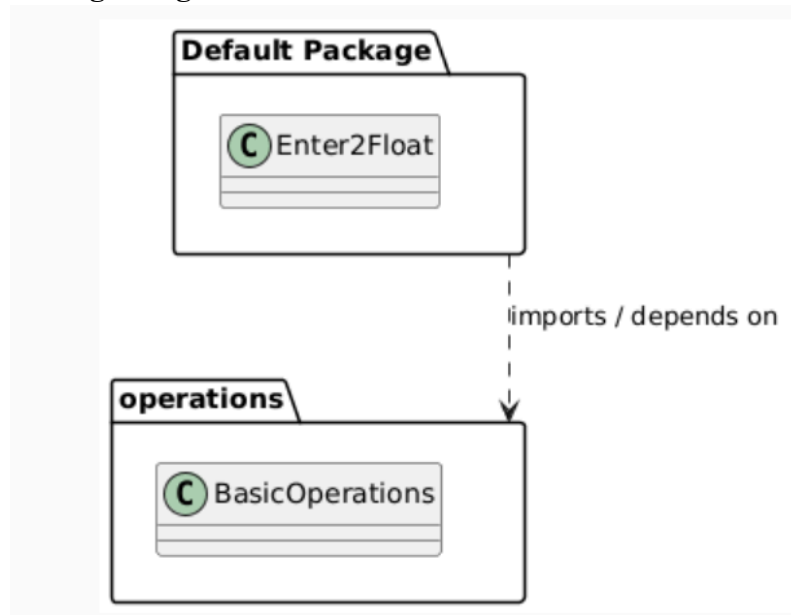
javac operations/BasicOperations.java Enter2Float.java

java Enter2Float 10.5 3.2

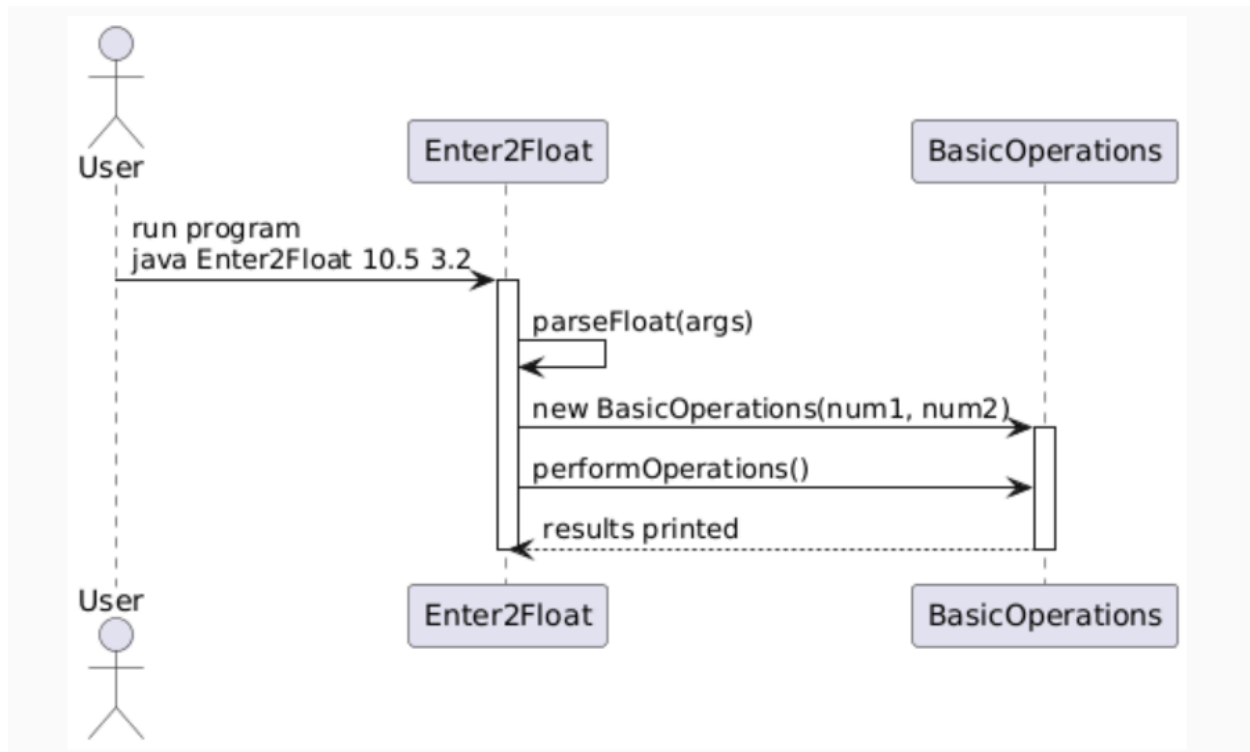
UML Class Diagram



Package Diagram



Sequence Diagram



Pgm3: To explore String and StringBuffer methods

project-folder/

```

├── Enter2Strings.java
├── textops/
│   ├── StringOperations.java
│   └── StringBufferOperations.java

```

textops.StringOperations.java

```
package textops;
```

```
public class StringOperations {
```

```
    private String str1;
    private String str2;
```

```
    public StringOperations(String str1, String str2) {
        this.str1 = str1;
        this.str2 = str2;
    }
```

```
    public void exploreStringMethods() {
        System.out.println("----- String Methods -----");
    }
```

```

        System.out.println("Length of str1: " + str1.length());
        System.out.println("Uppercase str1: " + str1.toUpperCase());
        System.out.println("Lowercase str1: " + str1.toLowerCase());
        System.out.println("Concatenation: " + str1.concat(str2));
        System.out.println("Equals: " + str1.equals(str2));
        System.out.println("Equals Ignore Case: " + str1.equalsIgnoreCase(str2));
        System.out.println("Contains str2 in str1: " + str1.contains(str2));
        System.out.println("Substring of str1 (0,3): " +
            (str1.length() >= 3 ? str1.substring(0, 3) : "Too short"));
        System.out.println("Replace 'a' with 'x' in str1: " + str1.replace('a', 'x'));
        System.out.println("Index of str2 in str1: " + str1.indexOf(str2));
        System.out.println("Trim str1: '" + str1.trim() + "'");
    }
}

```

textops.StringBufferOperations.java

```
package textops;
```

```

public class StringBufferOperations {

    private String str;

    public StringBufferOperations(String str) {
        this.str = str;
    }

    public void exploreStringBufferMethods(String appendStr) {
        System.out.println("\n----- StringBuffer Methods -----");

        // Create a StringBuffer object from str
        StringBuffer sb = new StringBuffer(str);

        System.out.println("Original StringBuffer: " + sb);

        // append() adds text to the end of the buffer (mutable)
        sb.append(appendStr);
        System.out.println("After append: " + sb);

        // insert() inserts text at a specified index (mutable)
        sb.insert(0, "Start-");
        System.out.println("After insert: " + sb);

        // replace() replaces characters between start and end index (mutable)
        sb.replace(0, 5, "Begin");
        System.out.println("After replace: " + sb);
    }
}

```

```

// delete() removes characters between start and end index (mutable)
sb.delete(0, 6);
System.out.println("After delete: " + sb);

// reverse() reverses the buffer content (mutable)
sb.reverse();
System.out.println("After reverse: " + sb);

// capacity() shows current buffer size; it automatically increases as needed
System.out.println("Capacity: " + sb.capacity());

// length() shows the number of characters currently in the buffer
System.out.println("Length: " + sb.length());

// NOTE: Unlike String, these operations modify the same object
// String operations (concat, replace, etc.) return new String objects
}
}

```

Enter2Strings.java

```

import textops.StringOperations;
import textops.StringBufferOperations;

public class Enter2Strings {

    public static void main(String[] args) {

        if (args.length != 2) {
            System.out.println("Usage: java Enter2Strings <str1> <str2>");
            return;
        }

        String str1 = args[0];
        String str2 = args[1];

        // Instantiate and call StringOperations
        StringOperations stringOps = new StringOperations(str1, str2);
        stringOps.exploreStringMethods();

        // Instantiate and call StringBufferOperations
        StringBufferOperations sbOps = new StringBufferOperations(str1);
        sbOps.exploreStringBufferMethods(str2);
    }
}

```

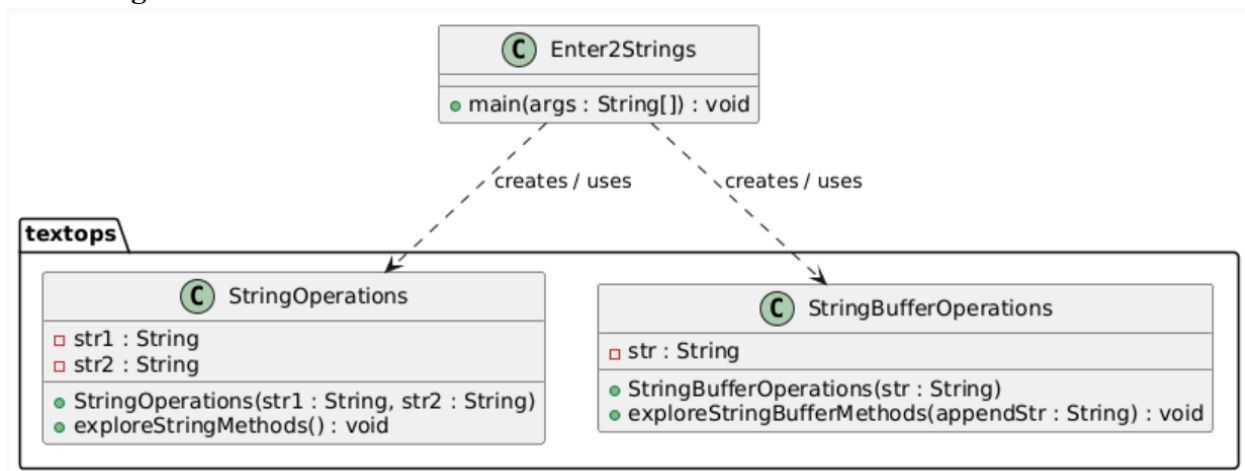
Compile & Run

```
proj-folder>javac textops/StringOperations.java textops/StringBufferOperations.java  
Enter2Strings.java
```

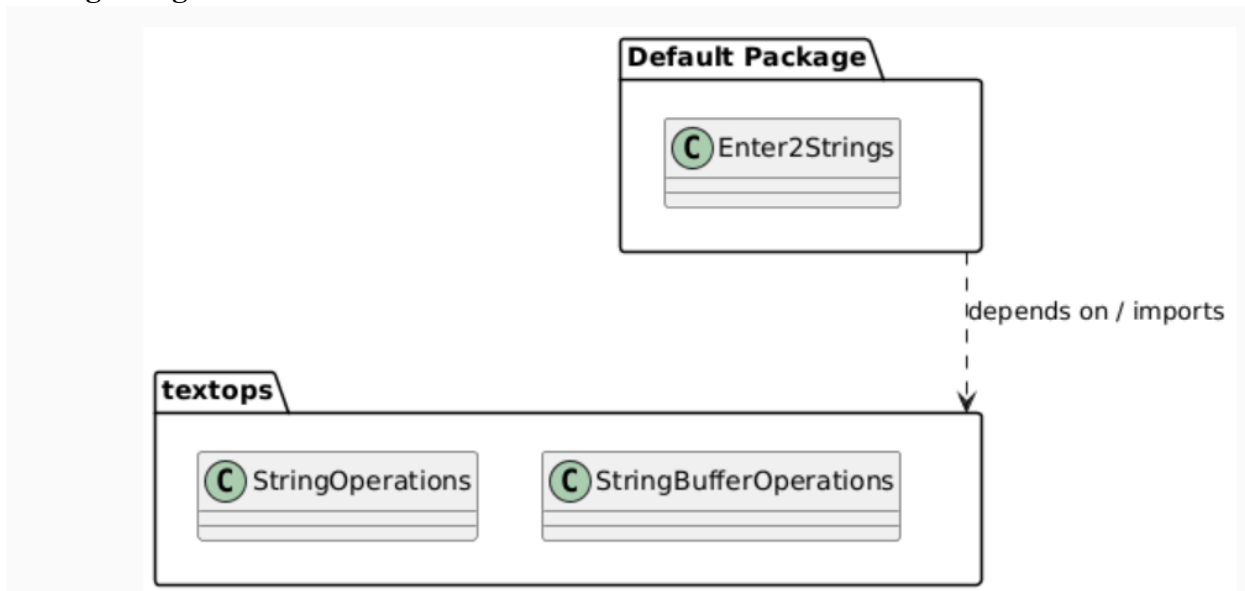
```
proj-folder>java Enter2Strings Hello World
```

UML Diagrams:

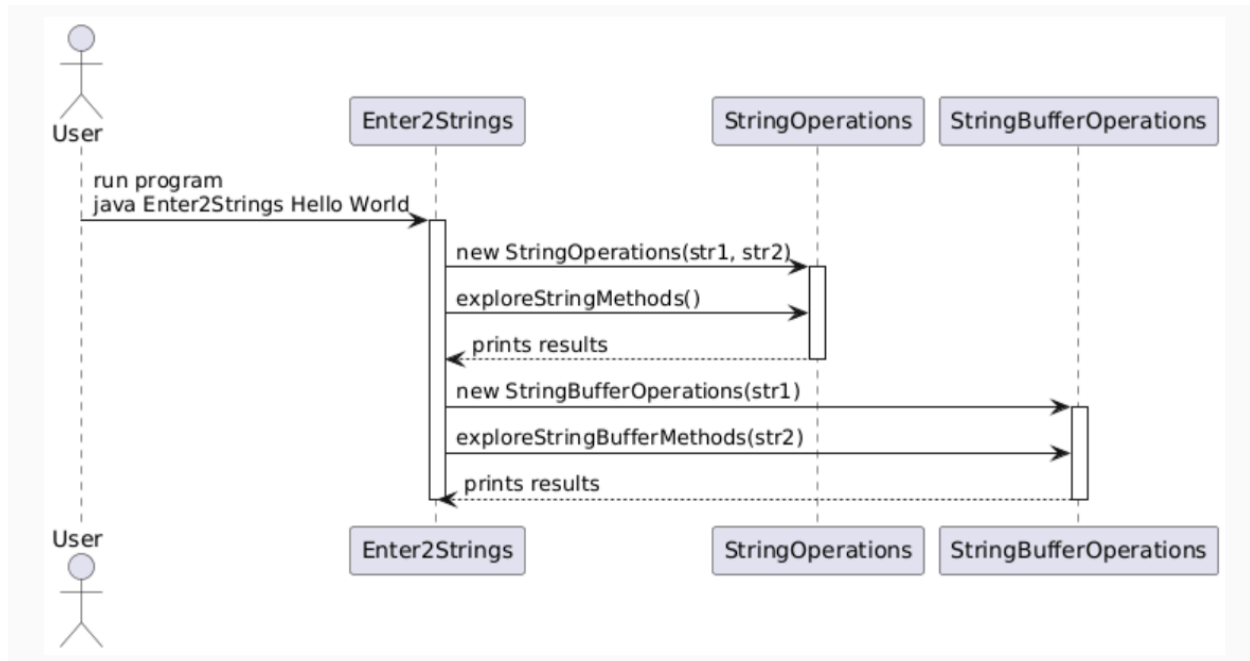
Class Diagram



Package Diagram



Sequence Diagram



Questions

1 Basic Java / Syntax

1. What is the difference between float and double in Java?
2. Why did we use `Float.parseFloat(args[0])` instead of directly assigning `args[0]`?
3. Can `Math` class methods be called on objects? Why or why not?
4. What happens if `args.length != 2`? How does the program handle it?

2 Math Class

5. Name 5 commonly used methods of the `Math` class and their purpose.
6. What is the difference between `Math.round()`, `Math.ceil()`, and `Math.floor()`?
7. How is `Math.random()` generated, and what is its range?
8. What does `Math.pow(a, b)` do if `b` is negative?
9. Can `Math.sqrt(-1)` be called? What happens?

3 OOP Concepts

10. Why did we move operations to a separate class (`BasicOperations`)?
11. What is the relationship between `Enter2Float` and `BasicOperations` in UML?
12. Why do we use a constructor in `BasicOperations`?
13. What is dependency, and where is it used in this program?

4 Command-Line Arguments

14. How do you pass command-line arguments when running a Java program?
15. What type of exception can occur when parsing a non-numeric string to float?
16. Can you run the program without arguments? What will happen?

Enter2Strings (String/StringBuffer Operations)

1 Basic Java / String

1. What is the difference between String and StringBuffer?
 2. Why is String immutable?
 3. Which methods of String create a new object and which modify the existing one?
 4. What happens if you call substring() with an invalid range?
-

2 String Methods

5. What is the difference between equals() and == for strings?
 6. Explain equalsIgnoreCase() with an example.
 7. What is the difference between trim() and strip()?
 8. What does indexOf() return if the substring is not found?
 9. How does replace() differ from replaceAll()?
-

3 StringBuffer Methods

10. What makes StringBuffer faster for multiple string modifications compared to String?
 11. Explain what append(), insert(), replace(), delete(), and reverse() do.
 12. What is the difference between StringBuffer and StringBuilder?
 13. What does capacity() mean in StringBuffer? How is it different from length()?
-

4 OOP / Package

15. Why did we create separate classes for String and StringBuffer operations?
 16. Explain the UML dependency relationship between Enter2Strings and the operation classes.
-

5 Command-Line Arguments & Runtime

19. What happens if you pass fewer or more than 2 strings?
 20. Could this program handle empty strings? What would happen?
-

4. Program to demonstrate:

Interface (Shape) with PI constant

Inheritance (Cube → Square, Sphere → Circle)

super() → call parent constructor

super.method() → reuse parent method

Dynamic polymorphism → Shape s = new Cube(3)

Static polymorphism → Square.area(5), Circle.area(4)

Javadoc, multiline, single-line comments for clarity

Shape Interface

File: shapes/Shape.java

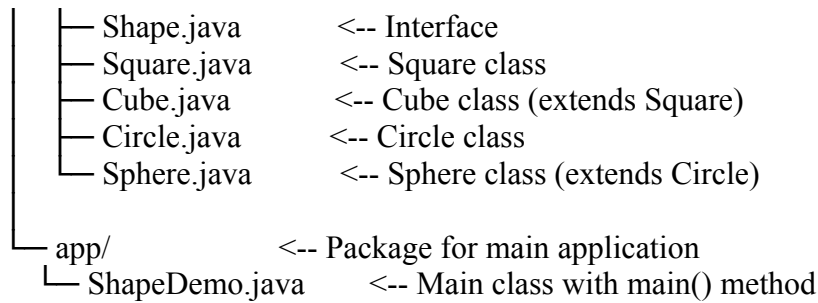
package shapes;

Folder Structure:

ProjectRoot/

├─ shapes/

<-- Package for all shape-related classes



```
/**
 * Interface representing geometric shapes.
 * All shapes must implement methods to calculate area and volume.
 */
public interface Shape {

    /** Constant value of PI */
    double PI = 3.14159;

    /**
     * Calculate the area of the shape.
     * @return area as double
     */
    double area();

    /**
     * Calculate the volume of the shape.
     * @return volume as double
     */
    double volume();
}
```

2 Square Class

File: shapes/Square.java

```
package shapes;
```

```
/**
 * Class representing a square.
 * Implements the Shape interface.
 */
public class Square implements Shape {

    /** Side length of the square */
    protected double side;

    /**
```

```

    * Constructor to initialize square side
    * @param s side length
    */
    public Square(double s) {
        side = s;
    }

    /**
     * Calculate area of the square
     * @return side^2
     */
    public double area() {
        return side * side;
    }

    // -----
    // Static overloaded methods
    // -----

    /**
     * Calculate area for a given side (double)
     * @param s side length
     * @return area
     */
    public static double area(double s) {
        return s * s;
    }

    /**
     * Calculate area for a given side (int)
     * @param s side length
     * @return area
     */
    public static double area(int s) {
        return s * s;
    }

    /**
     * Square does not have volume
     * @return 0
     */
    public double volume() {
        return 0;
    }
}

```

3 Cube Class

File: shapes/Cube.java

package shapes;

```
/**
 * Cube class extends Square
 * Demonstrates use of super() and super.area()
 */
public class Cube extends Square {

    /**
     * Constructor calls Square constructor
     * @param s side length
     */
    public Cube(double s) {
        super(s); // super() calls parent constructor
    }

    /**
     * Calculate surface area of cube
     * Uses one face area from Square
     * @return surface area
     */
    public double area() {
        double tmp = super.area(); // reuse Square area
        return 6 * tmp;
    }

    /**
     * Calculate cube volume
     * @return volume
     */
    public double volume() {
        return side * side * side;
    }
}
```

4 Circle Class

File: shapes/Circle.java

package shapes;

```
/**
 * Circle class implements Shape interface
 */
public class Circle implements Shape {
```

```

/** Radius of the circle */
protected double radius;

/**
 * Constructor
 * @param r radius
 */
public Circle(double r) {
    radius = r;
}

/**
 * Calculate area of the circle
 * @return PI * r^2
 */
public double area() {
    return PI * radius * radius;
}

// -----
// Static overloaded method
// -----

/**
 * Calculate area for a given radius
 * @param r radius
 * @return area
 */
public static double area(double r) {
    return PI * r * r;
}

/**
 * Circle has no volume
 * @return 0
 */
public double volume() {
    return 0;
}
}

```

5 Sphere Class

File: shapes/Sphere.java
package shapes;

```

/**

```

```

* Sphere class extends Circle
* Demonstrates reuse of Circle area using super.area()
*/
public class Sphere extends Circle {

    /**
     * Constructor calls Circle constructor
     * @param r radius
     */
    public Sphere(double r) {
        super(r); // super() call
    }

    /**
     * Calculate surface area of sphere
     * Reuses Circle's area method
     * @return surface area
     */
    public double area() {
        double tmp = super.area(); // circle area
        return 4 * tmp;
    }

    /**
     * Calculate volume of sphere
     * @return volume
     */
    public double volume() {
        return (4.0 / 3.0) * PI * radius * radius * radius;
    }
}

```

6 Main Program

File: app/ShapeDemo.java

```
package app;
```

```
// Import all classes from shapes package
import shapes.*;
```

```

/**
 * Main class to demonstrate shapes
 * Shows dynamic polymorphism and static overloading
 */
public class ShapeDemo {

    public static void main(String[] args) {

```

```

Shape s;

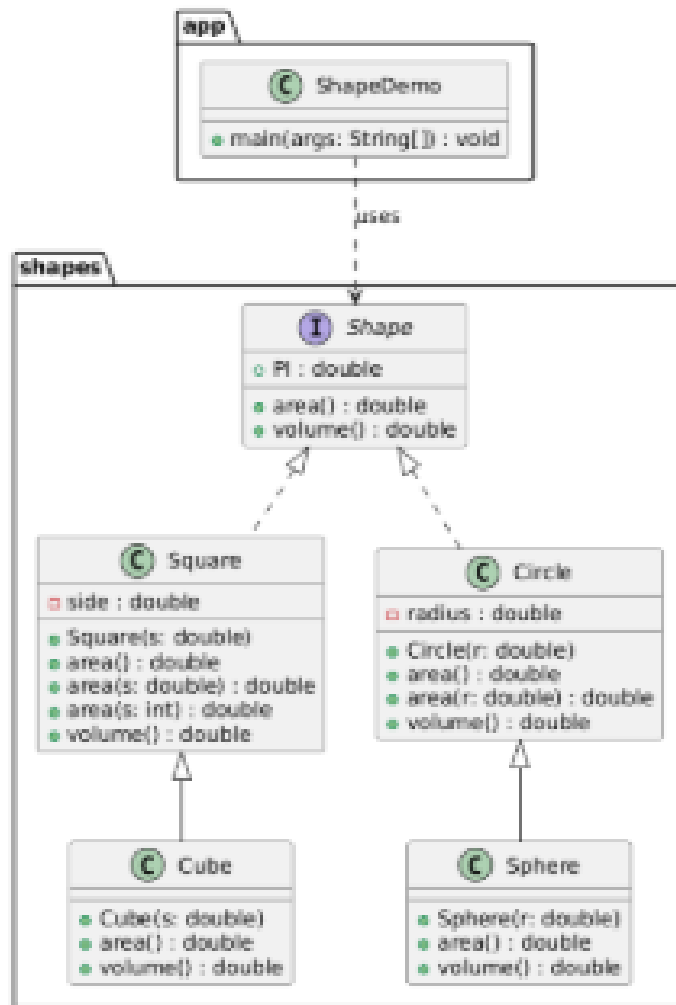
// -----
// Dynamic polymorphism examples
// -----
s = new Cube(3);
System.out.println("Cube Area = " + s.area());
System.out.println("Cube Volume = " + s.volume());

s = new Sphere(2);
System.out.println("Sphere Area = " + s.area());
System.out.println("Sphere Volume = " + s.volume());

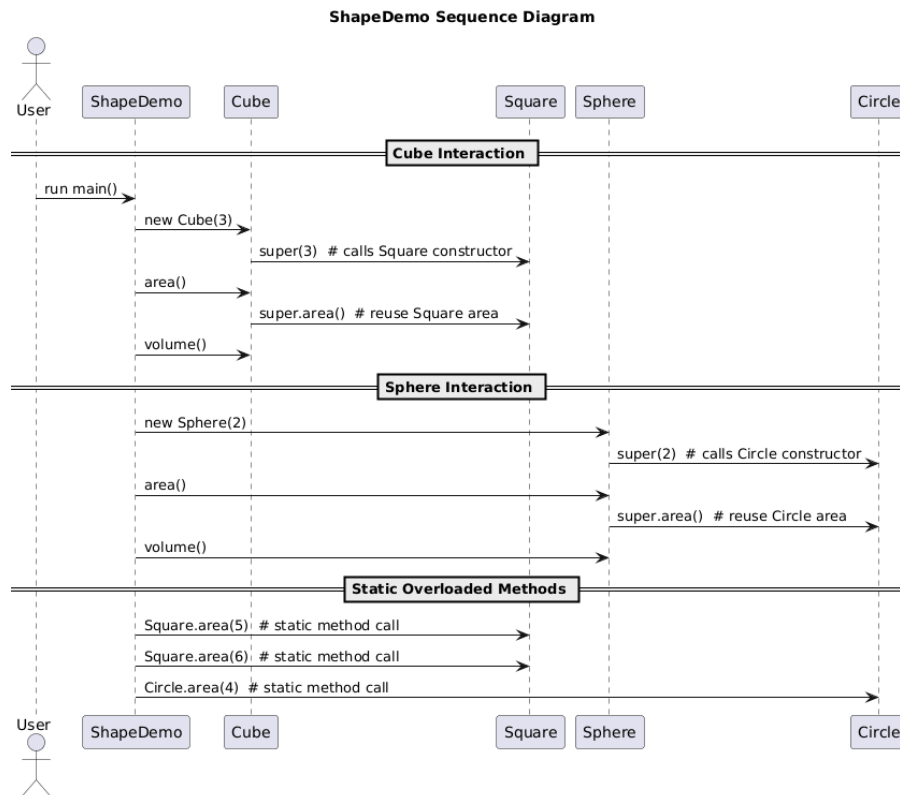
// -----
// Static polymorphism examples
// -----
System.out.println("Square Area (static) = " + Square.area(5));
System.out.println("Square Area (static int) = " + Square.area(6));
System.out.println("Circle Area (static) = " + Circle.area(4));
}
}

```

UML Class & Package Diagram:



Sequence Diagram:



Viva:

1. What is the purpose of having PI as a static final constant in the Shape interface?
2. How does the super() call in Cube and Sphere constructors work?
3. Why are Square.area() and Circle.area() defined as static overloaded methods?
4. Explain the difference between calling super.area() and this.area() in Cube or Sphere.
5. How does the ShapeDemo class demonstrate dynamic polymorphism?
6. What is the difference between super.area() and calling Square.area(5)?
7. What type of inheritance is used in Cube → Square and Sphere → Circle?
8. What is the difference between **single inheritance**, **multilevel inheritance**, and **hierarchical inheritance**? Which type is used here?
9. Explain the use of the protected access specifier for attributes like side and radius.
10. What is the difference between private, protected, public, and default (package-private) access in Java?
11. What does the keyword super do in Java? Give an example from Cube or Sphere.
12. How is this keyword different from super? Can it be used in Cube or Sphere?
13. Why is Shape declared as an interface and not a class? Could it have been an abstract class instead?
14. Can a subclass override a static method from its superclass? Why or why not?
15. What happens if you do not call super() explicitly in a subclass constructor?

16. Explain the difference between **dynamic polymorphism** and **static polymorphism** in the context of this program.
17. What is the effect of making area() a static method in Square or Circle? How is it called differently than instance methods?
18. What is method overloading? How is it different from method overriding in this program?
19. Can ShapeDemo access side of Cube directly if it is declared private in Square? Explain.
20. How does the Java compiler differentiate between Square.area() and Cube.area() when both exist?
21. Explain why the PI constant in the interface does not need to be redefined in subclasses.

5. Built in Exceptions and Static methods

```
public class ArithOp {

    public static void add(int a, int b) {
        int result = a + b;
        System.out.println("Addition = " + result);
    }

    public static void sub(int a, int b) {
        int result = a - b;
        System.out.println("Subtraction = " + result);
    }

    public static void mul(int a, int b) {
        int result = a * b;
        System.out.println("Multiplication = " + result);
    }

    /**
     * Performs division of two integers.
     * Handles division by zero using try-catch.
     *
     * @param a Dividend
     * @param b Divisor
     */
    public static void div(int a, int b) {
        try {
            // Attempt to divide a by b
            int result = a / b;
            System.out.println("Division = " + result);
        }
        catch (ArithmeticException e) {
            // This block executes if b is zero
            System.out.println("Error: Cannot divide by zero.");
        }
    }
}
```

```
    }  
    finally {  
        // This block always executes regardless of exception  
        System.out.println("Division operation finished.");  
    }  
}  
}
```

```
import java.util.Scanner;
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        System.out.print("Enter first number: ");  
        int num1 = sc.nextInt();
```

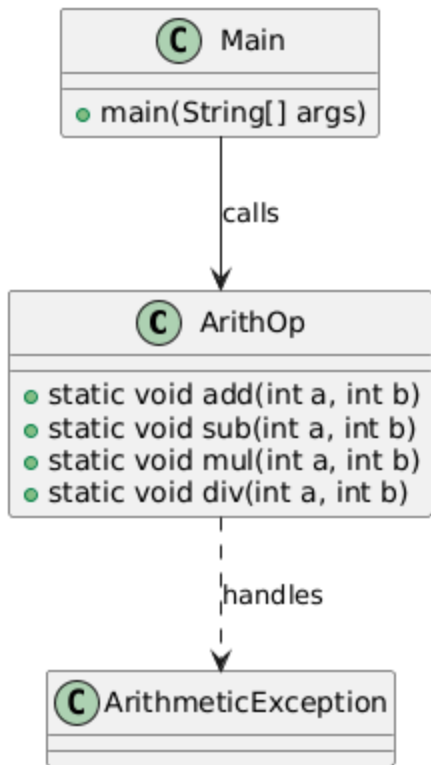
```
        System.out.print("Enter second number: ");  
        int num2 = sc.nextInt();
```

```
        // Call static methods directly using class name
```

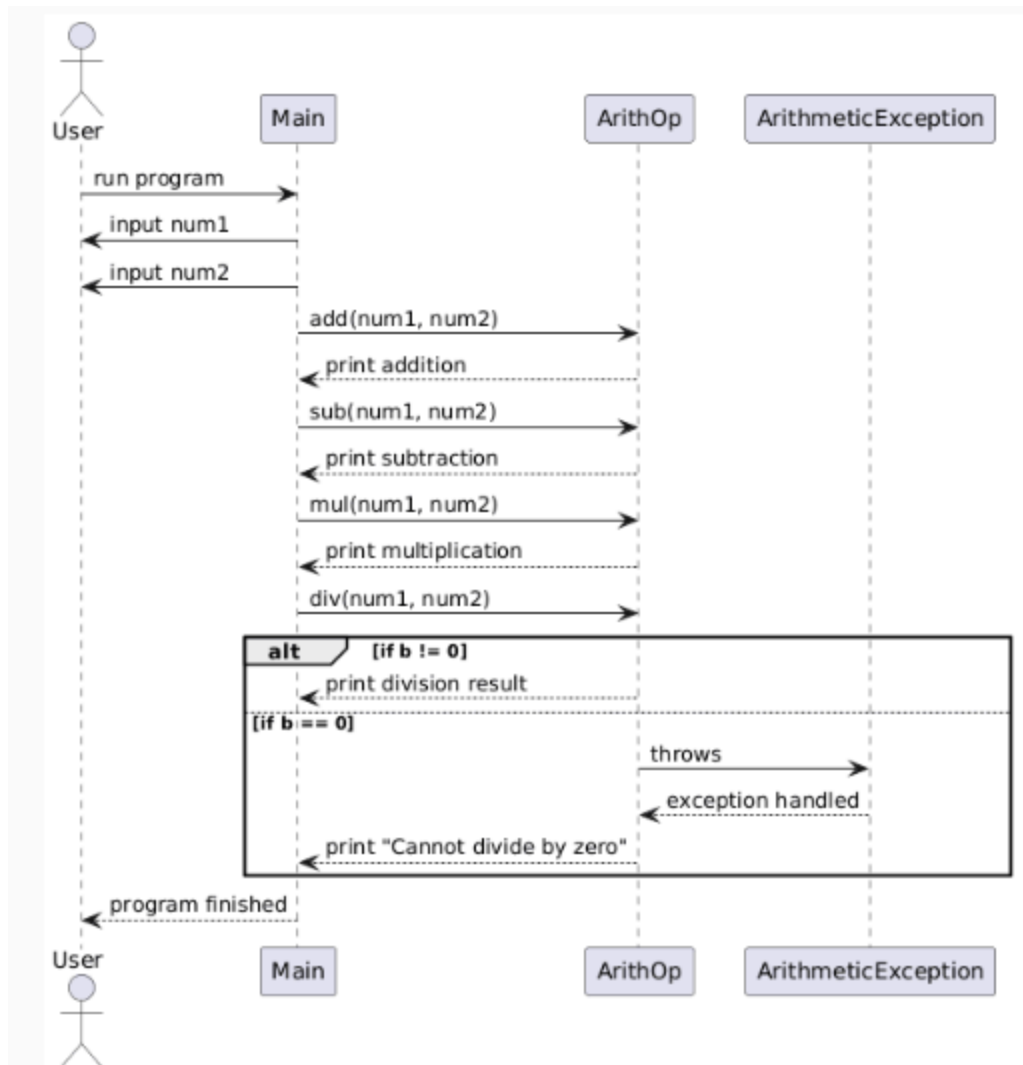
```
        ArithOp.add(num1, num2);  
        ArithOp.sub(num1, num2);  
        ArithOp.mul(num1, num2);  
        ArithOp.div(num1, num2);
```

```
        sc.close();
```

```
    }  
}
```

Sequence Diagram:



6. User-Defined (Custom) Exception

// File: InvalidAgeException.java

```

public class InvalidAgeException extends Exception {
    public InvalidAgeException(String message) {
        super(message);
    }
}

```

// File: UserRegistration.java

```

public class UserRegistration {

```

```

    /**

```

```

     * Logic and Validation moved here (The Calling Method)

```

```

    */

```

```

    public void processRegistration(String username, int age) {

```

```

System.out.println(">>> Processing registration for: " + username);

try {
    // Validation Logic (Moved here per request)
    if (age < 0) {
        throw new InvalidAgeException("Age cannot be negative (" + age + ").");
    }

    if (username == null || username.trim().isEmpty()) {
        throw new InvalidAgeException("Username is required.");
    }

    // If validation passes, call the registration/save action
    registerUser(username, age);

    System.out.println("STATUS: Success.");

} catch (InvalidAgeException e) {
    // Handle Failure Scenario
    System.err.println("STATUS: Failed - " + e.getMessage());
} finally {
    System.out.println("Finalizing request...\n");
}
}

/**
 * The simple action method
 */
private void registerUser(String username, int age) {
    System.out.println("System: Saving " + username + " (Age: " + age + ") to database.");
}
}

// File: Main.java
public class Main {
    public static void main(String[] args) {
        UserRegistration app = new UserRegistration();

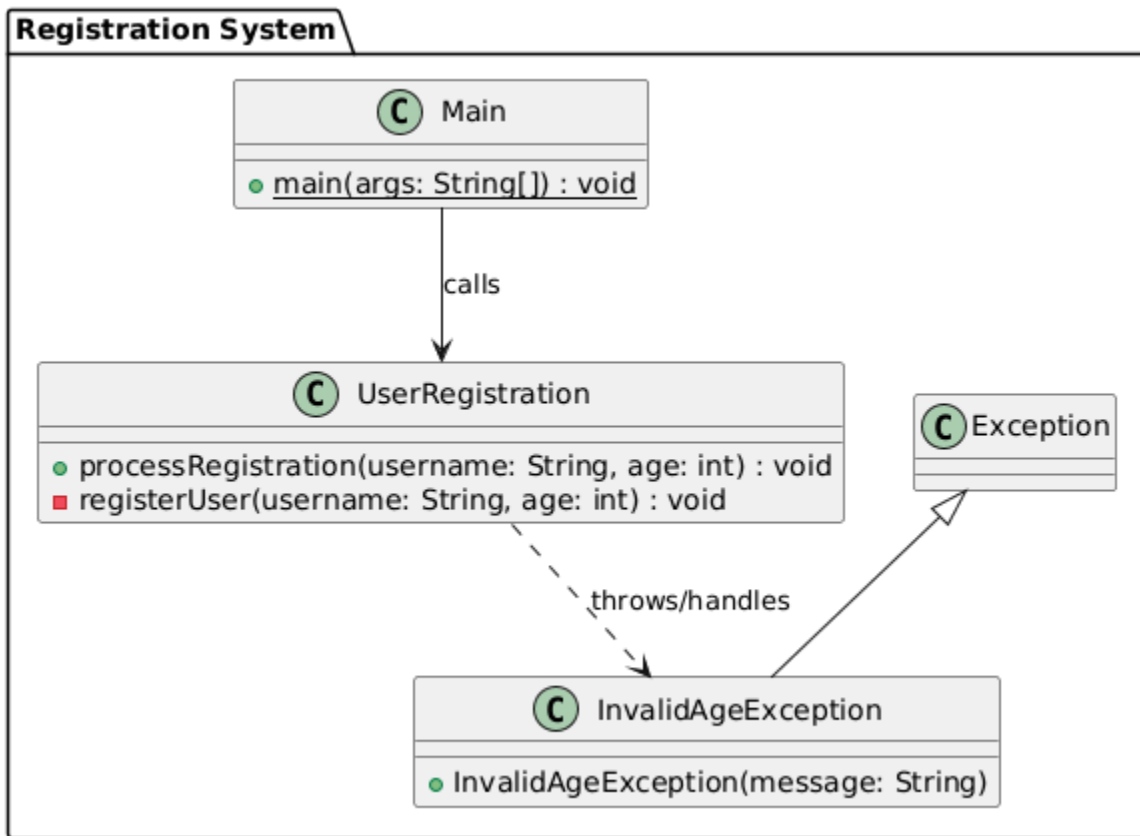
        // Scenario 1: Valid
        app.processRegistration("JavaExpert", 30);

        // Scenario 2: Invalid Age
        app.processRegistration("JuniorDev", -1);

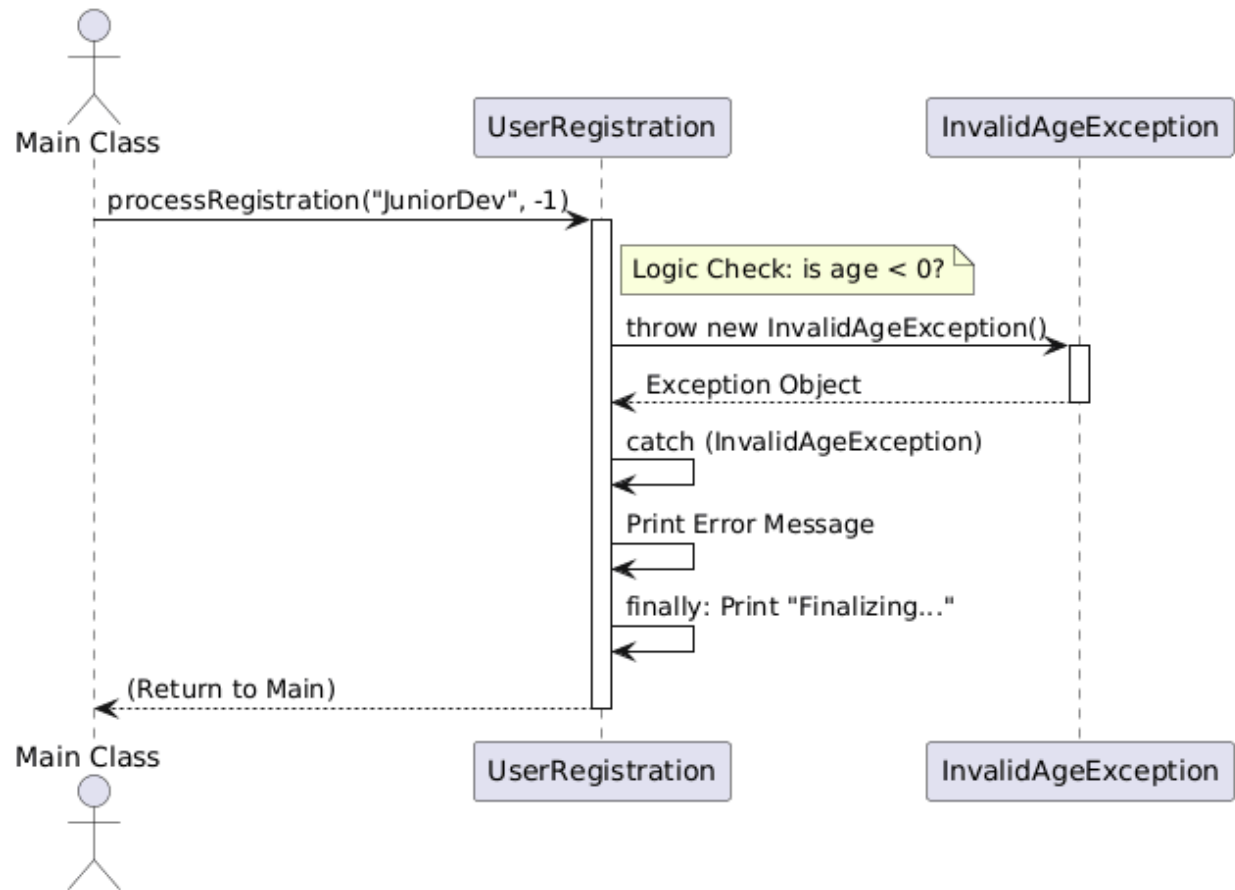
        // Scenario 3: Invalid Username
        app.processRegistration("", 20);
    }
}

```

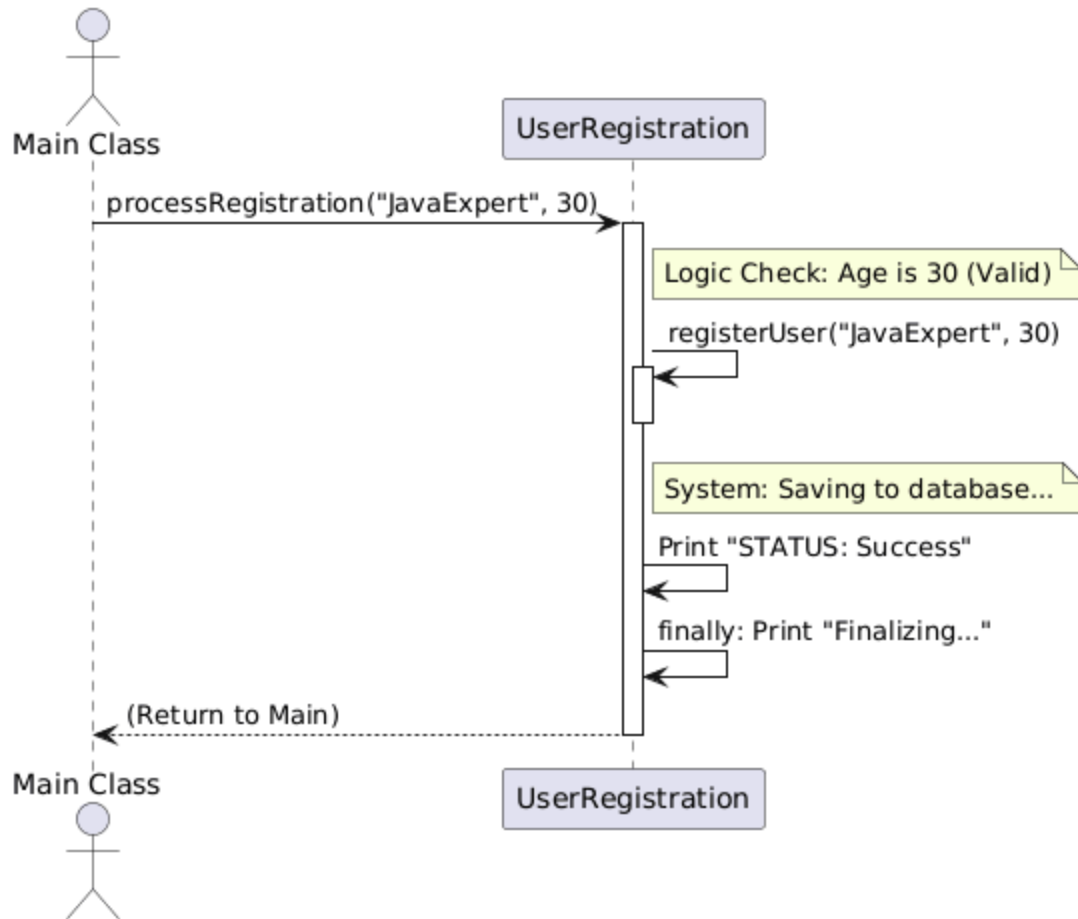
}
}



Sequence Diagram: Failure scenario



Sequence Diagram - Success Flow:



7. Multi threading:

// ILoop: Prints 1 to 50 with a '/' symbol

```

class ILoop implements Runnable {
    @Override
    public void run() {
        for (int i = 1; i <= 50; i++) {
            System.out.println("ILoop (/): " + i);
            try {
                // Short sleep to allow the other thread to take turns
                Thread.sleep(50);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }
    }
}

```

// JLoop: Prints 1 to 50 with a '\' symbol

```

class JLoop implements Runnable {

```

```

@Override
public void run() {
    for (int i = 1; i <= 50; i++) {
        System.out.println("JLoop (\\): " + i);
        try {
            Thread.sleep(50);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}

}

}

public class CountThreads {
    public static void main(String[] args) {
        System.out.println("--- Starting Concurrent Count ---\n");

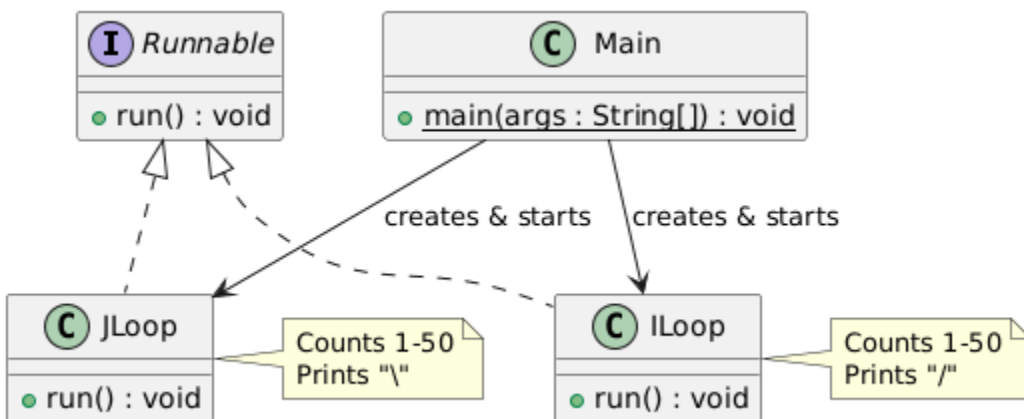
        Thread t1 = new Thread(new ILoop());
        Thread t2 = new Thread(new JLoop());

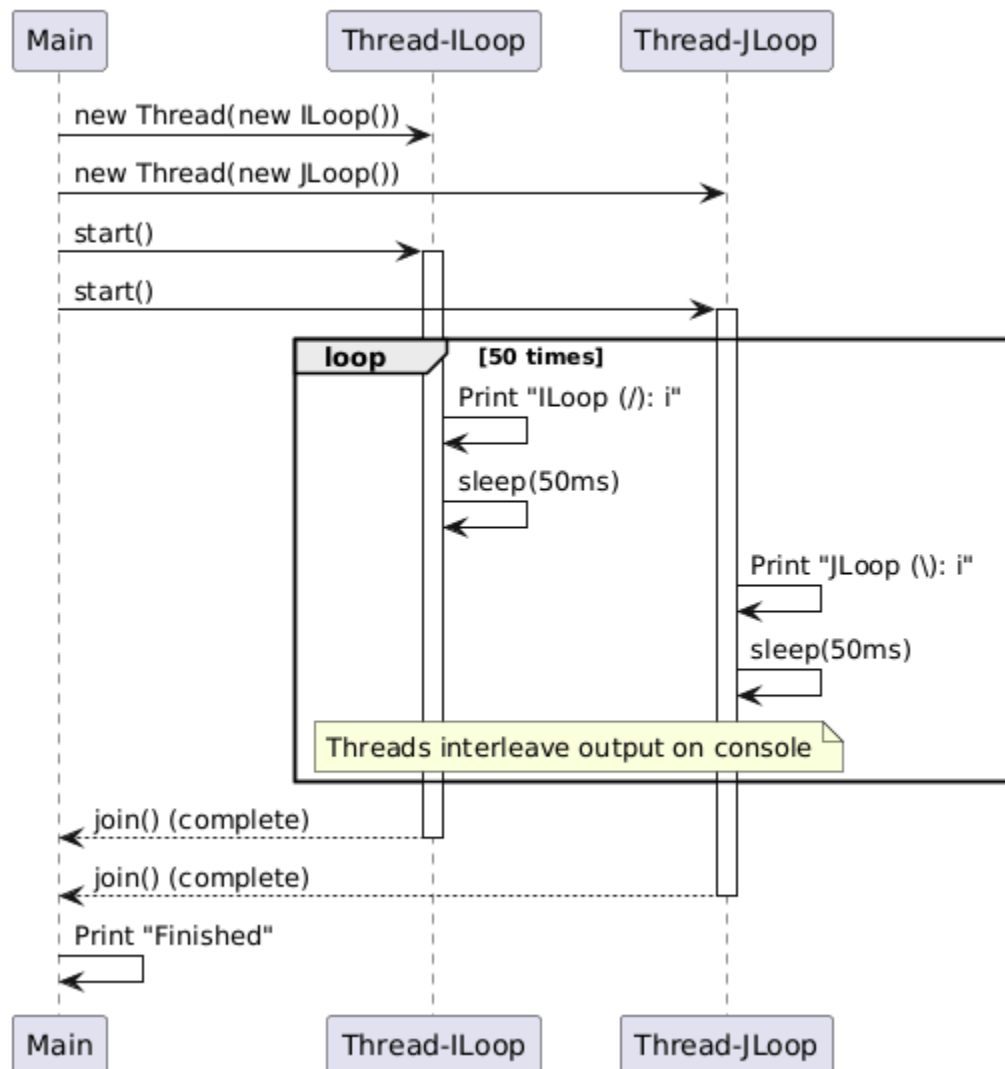
        t1.start();
        t2.start();

        try {
            t1.join();
            t2.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

        System.out.println("\n--- Both threads finished counting! ---");
    }
}

```





Viva Questions:

1. What is a thread?

A thread is a lightweight subprocess, the smallest unit of execution in a program.

2. What is multithreading?

Multithreading is the process of executing multiple threads simultaneously to improve performance.

3. What is the difference between process and thread?

Process	Thread
Heavyweight	Lightweight
Own memory	Shared memory

Slower

Faster

4. Why use multithreading?

- Better CPU utilization
- Faster execution
- Concurrent tasks

5. How to create a thread?

Two ways:

1. Extending **Thread** class
2. Implementing **Runnable** interface

6. Which is better: Thread or Runnable?

Runnable is better because:

- Supports multiple inheritance
- Separates task from thread

7. What are thread states?

- New
- Runnable
- Running
- Waiting / Blocked
- Terminated

8. What is **start()** method?

Starts a thread and calls **run()** internally.

10. What is **sleep()**?

Pauses thread execution for a specified time.

11. What is **join()**?

Waits for a thread to finish before continuing.

12. What is **yield()**?

Temporarily pauses and gives chance to other threads.

13. What is **isAlive()**?

Checks whether a thread is still running.

14. What is synchronization?

It is a process of controlling access to shared resources.

15. Why is synchronization needed?

To avoid:

- Data inconsistency
- Race conditions

16. What is a race condition?

When multiple threads access shared data simultaneously causing unexpected results.

17. What is a synchronized method?

A method that allows only one thread at a time.

```
synchronized void display() { }
```

18. What is inter-thread communication?

Threads communicate with each other using methods like:

- `wait()`
- `notify()`
- `notifyAll()`

19. What is `wait()`?

Makes a thread wait until another thread notifies it.

20. What is `notify()`?

Wakes up one waiting thread.

21. What is `notifyAll()`?

Wakes up all waiting threads.

22. What is deadlock?

A situation where two or more threads wait forever for each other's resources.

25. What is daemon thread?

A background thread that runs in the background (e.g., garbage collector).

26. What is thread pool?

A group of reusable threads used to perform tasks efficiently.

8. File Operations

```
import java.util.Scanner;
```

```
import java.io.*;
```

```
import java.nio.file.*;
```

```
public class FileMenuSystem {
```

```
    private static final String FILE_PATH = "userdata.txt";
```

```
    public static void main(String[] args) {
```

```
        Scanner scanner = new Scanner(System.in);
```

```
        boolean running = true;
```

```
        while (running) {
```

```
            System.out.println("\n--- FILE MANAGER MENU ---");
```

```

        System.out.println("1. Read File");
        System.out.println("2. Write to File");
        System.out.println("3. Exit");
        System.out.print("Select an option: ");

        String choice = scanner.nextLine();

        switch (choice) {
            case "1":
                readFile();
                break;
            case "2":
                writeFile(scanner);
                break;
            case "3":
                System.out.println("Exiting program...");
                running = false;
                break;
            default:
                System.out.println("Invalid option. Please try again.");
        }
    }
    scanner.close();
}

private static void readFile() {
    try {
        File file = new File(FILE_PATH);
        if (!file.exists()) {
            System.out.println("Error: File does not exist yet. Write something first!");
            return;
        }
        String content = Files.readString(Path.of(FILE_PATH));
        System.out.println("\n--- FILE CONTENT ---");
        System.out.println(content);
        System.out.println("-----");
    } catch (IOException e) {
        System.err.println("Failed to read: " + e.getMessage());
    }
}

private static void writeFile(Scanner scanner) {
    System.out.print("Enter text to save to file: ");
    String data = scanner.nextLine();

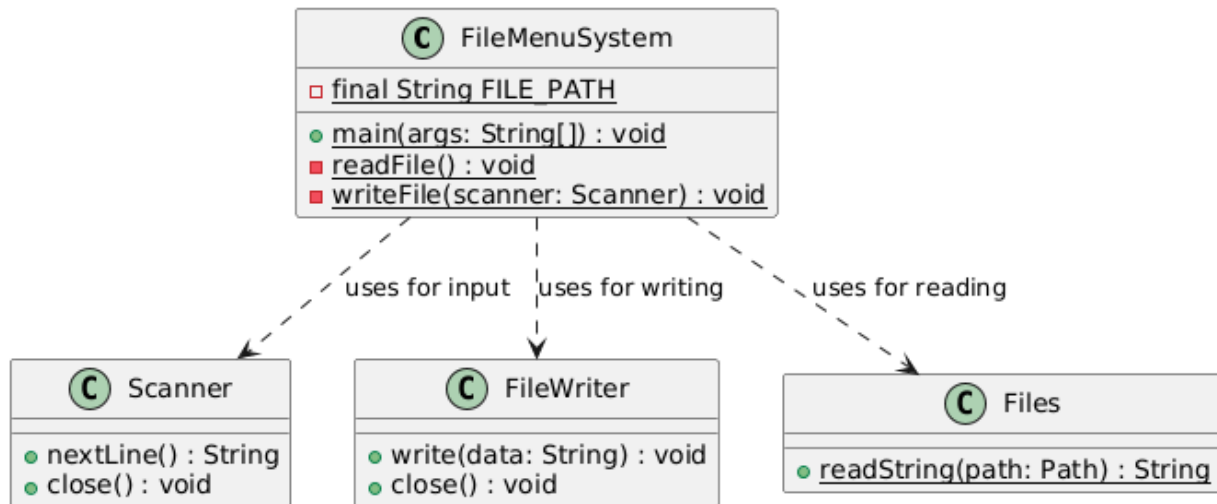
    try (FileWriter writer = new FileWriter(FILE_PATH, true)) { // 'true' enables appending

```

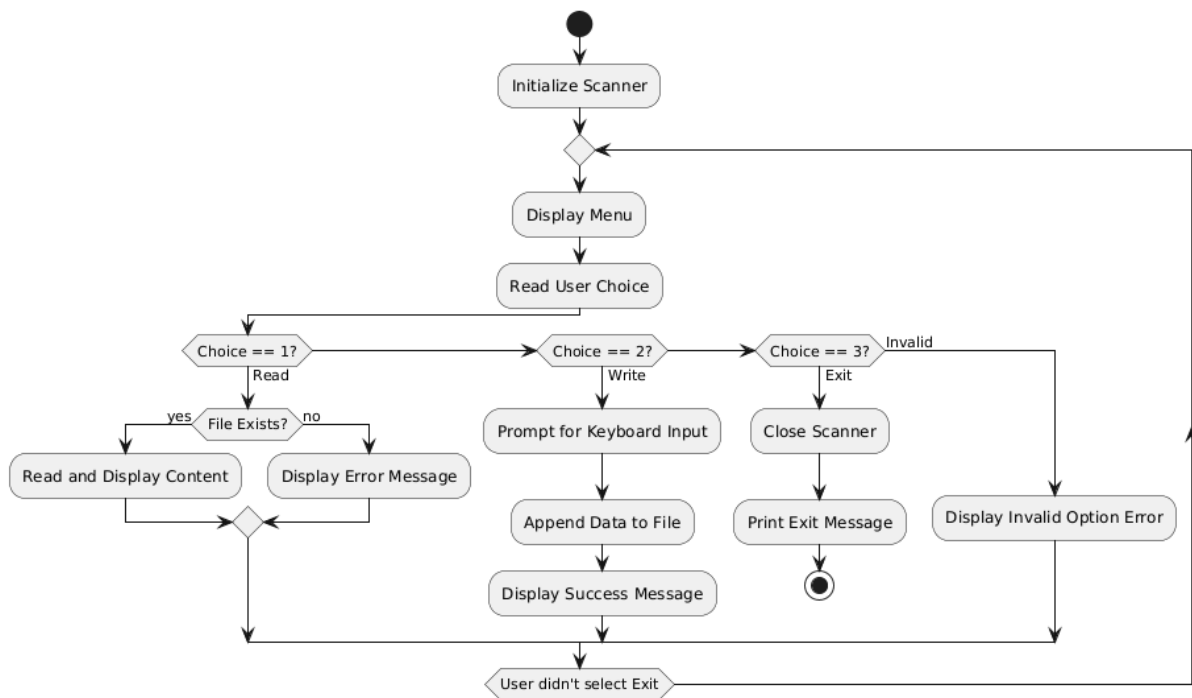
```

        writer.write(data + System.lineSeparator());
        System.out.println("Success: Data saved to " + FILE_PATH);
    } catch (IOException e) {
        System.err.println("Failed to write: " + e.getMessage());
    }
}
}
}

```



Activity Diagram:



Viva:

1. What is I/O in Java?

I/O (Input/Output) is used to read data from input sources and write data to output destinations.

2. What is a stream?

A stream is a flow of data between a source and a destination.

3. What are the types of streams?

- Input Stream → Reads data
- Output Stream → Writes data

4. What are byte streams?

Used to handle binary data (images, audio, etc.)

Examples:

- FileInputStream
- FileOutputStream

5. What are character streams?

Used to handle text data

Examples:

- FileReader
- FileWriter

6. What is InputStream?

Base class for reading byte data.

7. What is OutputStream?

Base class for writing byte data.

8. Common byte stream classes

- FileInputStream
- FileOutputStream
- BufferedInputStream
- BufferedOutputStream

9. What is Reader?

Base class for reading characters.

10. What is Writer?

Base class for writing characters.

11. Common character stream classes

- FileReader
- FileWriter
- BufferedReader
- BufferedWriter

12. What is buffering?

Temporarily storing data in memory to improve performance.

13. Why use `BufferedReader`?

- Faster reading
- Supports `readLine()` method

14. What is `read()`?

Reads data from a stream.

15. What is `write()`?

Writes data to a stream.

16. What is `close()`?

Closes the stream and releases resources.

17. What is `flush()`?

Forces data to be written immediately.

18. What is `File` class?

Used to create, delete, and get information about files.

19. Difference between `FileReader` and `FileInputStream`?

<code>FileReader</code>	<code>FileInputStream</code>
-------------------------	------------------------------

Character stream	Byte stream
------------------	-------------

For text	For binary
----------	------------

20. What is serialization?

Converting an object into a byte stream.

21. What is deserialization?

Converting byte stream back to object.

22. What is `IOException`?

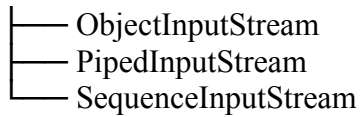
An exception that occurs during input-output operations.

25. What is `PrintWriter`?

Used for formatted output.

Sample Input stream hierarchy:

```
InputStream (abstract)
├── FileInputStream
├── ByteArrayInputStream
├── BufferedInputStream
└── DataInputStream
```



9. Exploring java.util collections (DataStructures) and javadoc: Menu driven Student Management System

```
import java.util.Scanner;
import java.util.Vector;
import java.util.Hashtable;
import java.util.Iterator;

/**
 * Represents a Student with registration number, name, and marks.
 * Demonstrates OOP concepts and uses Hashtable for subject → marks mapping.
 */
class Student {
    int regNo;
    String name;
    Hashtable<String, Integer> marks;

    /**
     * Constructs a Student object with registration number and name.
     * Initializes the marks Hashtable.
     *
     * @param regNo registration number of the student
     * @param name name of the student
     */
    Student(int regNo, String name) {
        this.regNo = regNo;
        this.name = name;
        marks = new Hashtable<>();
    }

    /**
     * Adds a subject and its marks to the student's record.
     *
     * @param subject name of the subject
     * @param mark marks obtained in the subject
     * @throws IllegalArgumentException if mark is negative or greater than 100
     */
    void addMarks(String subject, int mark) throws IllegalArgumentException {
        if (mark < 0 || mark > 100) {
            throw new IllegalArgumentException("Mark must be between 0 and 100.");
        }
        marks.put(subject, mark);
    }
}
```

```

    }

    /**
     * Calculates the average marks of the student.
     *
     * @return average of all marks as double
     */
    double calculateAverage() {
        int sum = 0;
        for (int m : marks.values()) {
            sum += m;
        }
        return marks.size() == 0 ? 0.0 : (double) sum / marks.size();
    }

    /**
     * Displays student details including registration number, name, subjects, and average marks.
     */
    void display() {
        System.out.println("RegNo: " + regNo);
        System.out.println("Name: " + name);
        System.out.println("Marks: " + marks);
        System.out.println("Average: " + calculateAverage());
        System.out.println("-----");
    }
}

/**
 * Menu-driven Student Management System.
 * Features:
 * - Register new student
 * - Add marks
 * - Search student by registration number
 * - Display all students
 * - Unregister student
 */
public class StudentMenuSystem {

    /**
     * Searches for a student in the Vector by registration number.
     *
     * @param students Vector of Student objects
     * @param regNo    registration number to search
     * @return the Student object if found; null otherwise
     */
    public static Student searchStudent(Vector<Student> students, int regNo) {

```



```

        for (Student s : students) {
            if (s.regNo == regNo) {
                return s;
            }
        }
        return null;
    }
}

/**
 * Main method for the menu-driven Student Management System.
 * Provides a looped menu to perform operations like register, add marks, search, display,
unregister, and exit.
 *
 * @param args command-line arguments
 * @throws IllegalArgumentException if marks entered are invalid
 */
public static void main(String[] args) throws IllegalArgumentException {
    Scanner sc = new Scanner(System.in);
    Vector<Student> students = new Vector<>();
    int choice;

    do {
        System.out.println("\n--- Student Management System ---");
        System.out.println("1. Register New Student");
        System.out.println("2. Add Marks");
        System.out.println("3. Search Student");
        System.out.println("4. Display All Students");
        System.out.println("5. Unregister Student");
        System.out.println("6. Exit");
        System.out.print("Enter your choice: ");
        choice = sc.nextInt();
        sc.nextLine(); // consume newline

        switch (choice) {
            case 1:
                // Register new student
                System.out.print("Enter RegNo: ");
                int regNo = sc.nextInt();
                sc.nextLine();
                System.out.print("Enter Name: ");
                String name = sc.nextLine();

                if (searchStudent(students, regNo) != null) {
                    System.out.println("Student with this RegNo already exists!");
                } else {
                    students.add(new Student(regNo, name));
                }
            }
        }
    } while (choice != 6);
}

```

```
        System.out.println("Student registered successfully.");
    }
    break;
```

case 2:

```
    // Add marks
    System.out.print("Enter RegNo to add marks: ");
    int regSearch = sc.nextInt();
    sc.nextLine();
    Student s = searchStudent(students, regSearch);
    if (s != null) {
        System.out.print("Enter subject name: ");
        String subject = sc.nextLine();
        System.out.print("Enter marks: ");
        int mark = sc.nextInt();
        sc.nextLine();
        try {
            s.addMarks(subject, mark);
            System.out.println("Marks added successfully.");
        } catch (IllegalArgumentException e) {
            System.out.println("Error: " + e.getMessage());
        }
    } else {
        System.out.println("Student not found!");
    }
    break;
```

case 3:

```
    // Search student
    System.out.print("Enter RegNo to search: ");
    int searchReg = sc.nextInt();
    sc.nextLine();
    Student found = searchStudent(students, searchReg);
    if (found != null) {
        System.out.println("\nStudent Found:");
        found.display();
    } else {
        System.out.println("Student not found!");
    }
    break;
```

case 4:

```
    // Display all students
    if (students.isEmpty()) {
        System.out.println("No students registered.");
    } else {
```

```

        System.out.println("\n--- All Students ---");
        Iterator<Student> it = students.iterator();
        while (it.hasNext()) {
            Student st = it.next();
            st.display();
        }
    }
    break;

case 5:
    // Unregister student
    System.out.print("Enter RegNo to unregister: ");
    int delReg = sc.nextInt();
    sc.nextLine();
    Student removeStu = searchStudent(students, delReg);
    if (removeStu != null) {
        students.remove(removeStu);
        System.out.println("Student unregistered successfully.");
    } else {
        System.out.println("Student not found!");
    }
    break;

case 6:
    System.out.println("Exiting program...");
    break;

default:
    System.out.println("Invalid choice! Try again.");
}

} while (choice != 6);

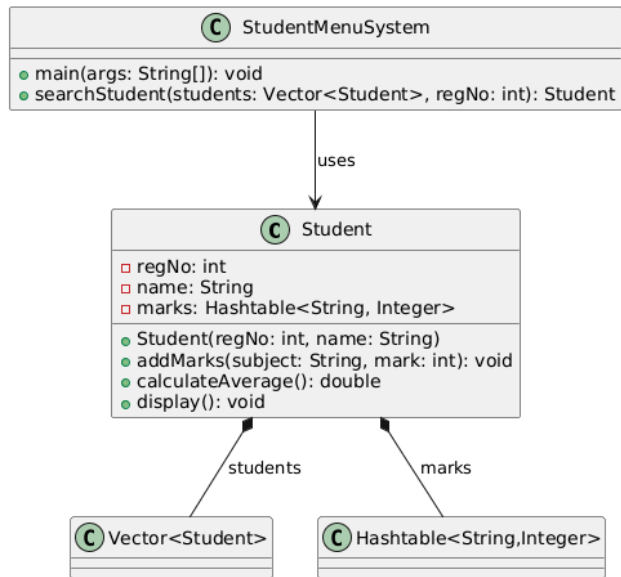
sc.close();
}
}

```

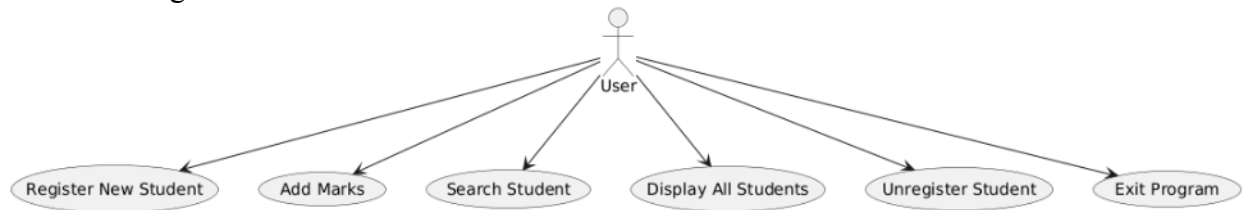
To generate javadoc in doc folder

```
d:\java>javadoc -d doc StudentMenuSystem.java Student.java
```

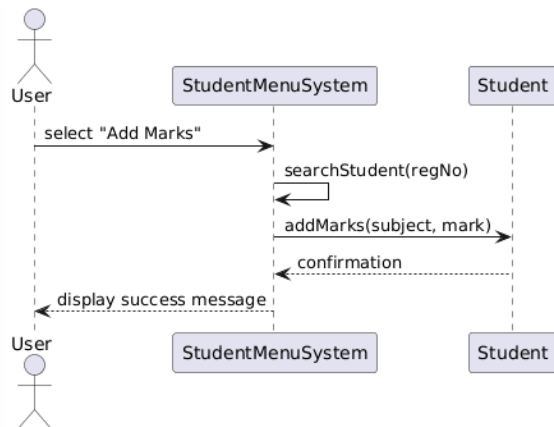
Class Diagram:



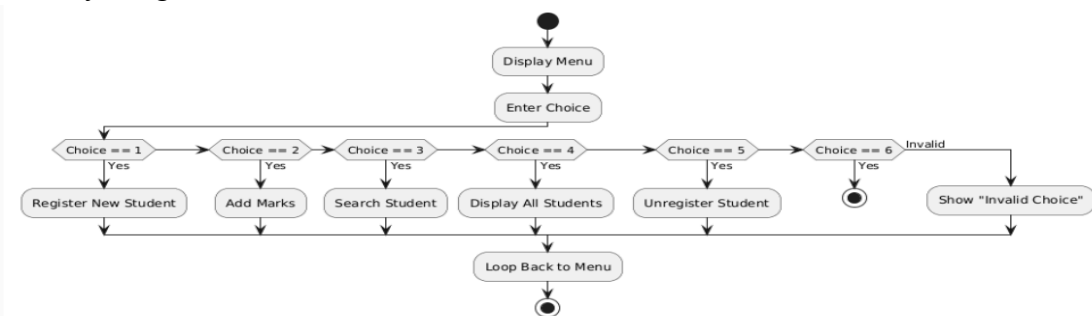
Use Case Diagram:



Sequence Diagram (addMarks())



Activity Diagram:



Viva:

Collection Type	Implementation	Allows Duplicates?	Maintains Order?	Sorted?	Key Features
List	ArrayList	Yes	Insertion order	No	Fast random access, dynamic array
	LinkedList	Yes	Insertion order	No	Efficient insert/delete, doubly linked
	Vector	Yes	Insertion order	No	Thread-safe (legacy)
Set	HashSet	No	No	No	Fast operations, uses hashing
	LinkedHashSet	No	Insertion order	No	Maintains order + uniqueness
	TreeSet	No	No	Yes	Sorted, uses Red-Black tree
Queue	PriorityQueue	Yes	No	Yes (priority)	Elements ordered by priority
	LinkedList	Yes	FIFO	No	Can act as queue/deque
Map	HashMap	No (keys)	No	No	Key-value pairs, fast lookup
	LinkedHashMap	No (keys)	Insertion order	No	Predictable iteration order
	TreeMap	No (keys)	No	Yes	Sorted keys, navigable map
	Hashtable	No (keys)	No	No	Thread-safe (legacy)

- **Uniqueness:**
 - List → allows duplicates
 - Set → no duplicates
 - Map → keys are unique, values can repeat
- **Ordering:**

- ArrayList, LinkedList, LinkedHashSet, LinkedHashMap preserve insertion order
- TreeSet, TreeMap sort elements
- **Performance:**
 - HashSet / HashMap → fastest ($O(1)$ average)
 - TreeSet / TreeMap → $O(\log n)$ due to sorting

JavaDoc Basics

1. What is JavaDoc and why do we use it?
2. What is the difference between inline comments and JavaDoc?
3. Which JavaDoc tags have you used and why?
4. Explain the addMarks method JavaDoc.
 - @param subject → the subject name
 - @param mark → marks obtained
 - @throws IllegalArgumentException → if mark is invalid
 - Shows input validation and proper API documentation
5. How do you generate JavaDoc?
6. What does -d option do in JavaDoc command?
7. What files are generated by JavaDoc?

10. Sample GUI with Event Handling: Login Page using javax.swing and keyboard listener

Program:

```
// Importing required Swing and AWT libraries
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

// Creating a class that extends JFrame (window)
// and implements ActionListener (for button click handling)
public class LoginPage extends JFrame implements ActionListener {

    // Declaring GUI components
    JLabel userLabel, passLabel;    // Labels for username and password
    JTextField userText;            // Text field for username input
    JPasswordField passText;        // Password field (hides input)
    JButton loginButton;            // Login button

    // Constructor to set up the GUI
    LoginPage() {
        setTitle("Login Page");    // Set window title
        setSize(350, 200);        // Set window size
        setLayout(null);           // Use absolute positioning
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); // Close app on exit

        // Username Label
```

```

userLabel = new JLabel("Username:");
userLabel.setBounds(30, 30, 80, 25); // Set position and size
add(userLabel); // Add to frame

// Username Text Field
userText = new JTextField();
userText.setBounds(120, 30, 150, 25);
add(userText);

// Password Label
passLabel = new JLabel("Password:");
passLabel.setBounds(30, 70, 80, 25);
add(passLabel);

// Password Field
passText = new JPasswordField();
passText.setBounds(120, 70, 150, 25);
add(passText);

// Login Button
loginButton = new JButton("Login");
loginButton.setBounds(120, 110, 100, 30);
loginButton.addActionListener(this); // Register button click event
add(loginButton);

setVisible(true); // Make the frame visible
}

// This method is called when the button is clicked
public void actionPerformed(ActionEvent e) {
    // Get entered username
    String username = userText.getText();

    // Get entered password (converted from char array to String)
    String password = new String(passText.getPassword());

    // Simple validation (hardcoded values)
    if (username.equals("admin") && password.equals("1234")) {
        // Show success message
        JOptionPane.showMessageDialog(this, "Login Successful!");
    } else {
        // Show error message
        JOptionPane.showMessageDialog(this, "Invalid Username or Password");
    }
}
}

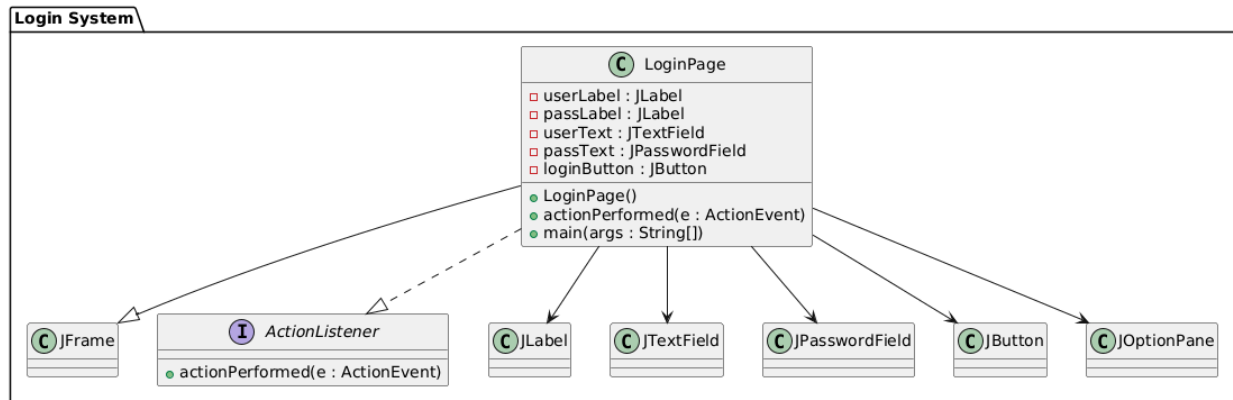
```

```

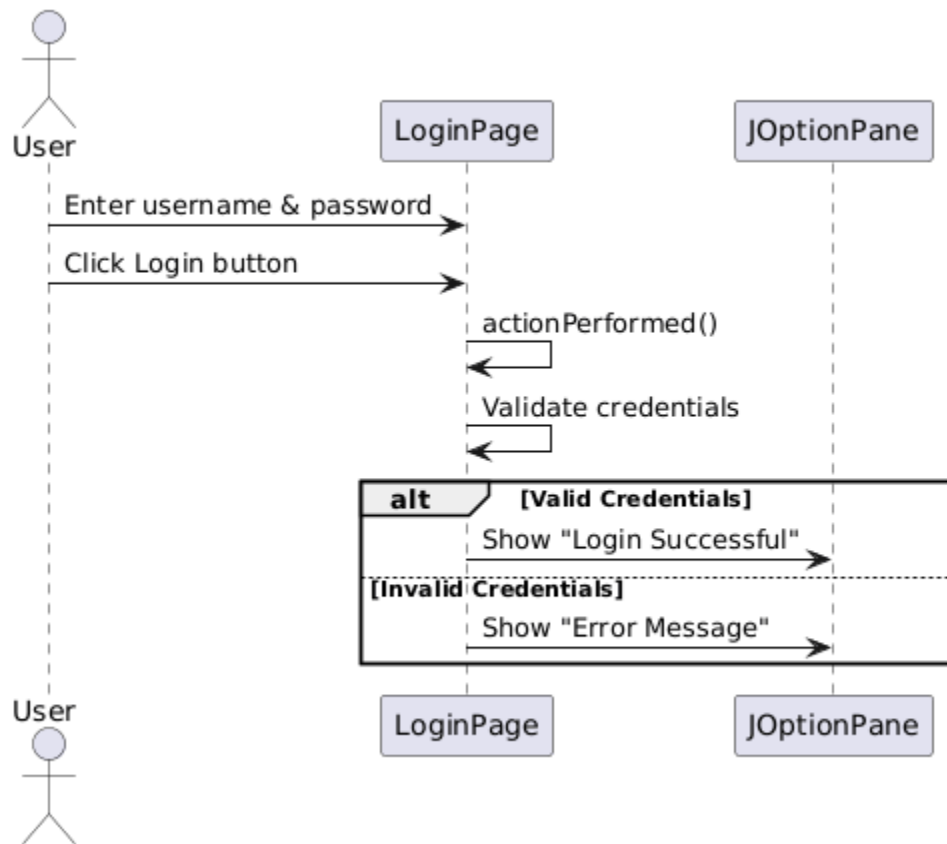
// Main method - entry point of the program
public static void main(String[] args) {
    new LoginPage(); // Create and display the login page
}
}

```

Class Diagram:



Sequence Diagram:



1. What is a GUI?

A **Graphical User Interface** allows users to interact with applications using visual elements like buttons, text fields, and windows instead of text commands.

2. What are components in GUI?

Components are **visual elements** like:

- Button
- Label
- TextField
- Checkbox

3. What is a container?

A container holds and organizes components.

Examples:

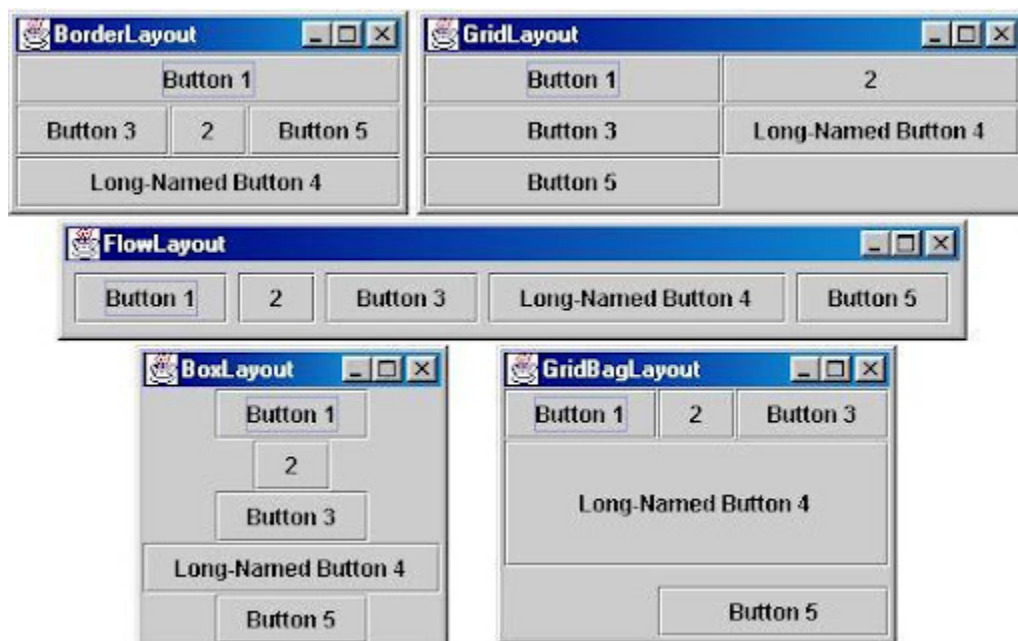
- Frame
- Panel

4. What is layout manager?

It controls the **arrangement of components** inside a container.

Examples:

- FlowLayout
- BorderLayout
- GridLayout



5. What is event-driven programming?

A programming style where execution depends on **user actions** like clicks or key presses.

6. What is AWT?

AWT is Java's **old GUI toolkit** that uses **native OS components**.

7. What are heavyweight components?

Components that depend on OS (platform-dependent).

AWT components are heavyweight.

8. Give examples of AWT components.

- Button
- Label
- TextField
- Frame

9. What are limitations of AWT?

- Platform dependent
- Limited features
- Less flexible UI

10. What is Swing?

Swing is an **advanced Java GUI toolkit** built on top of AWT.

11. What are lightweight components?

Components that are **platform-independent** and written in Java.

Swing components are lightweight.

12. Difference between AWT and Swing?

Feature	AWT	Swing
Type	Heavyweight	Lightweight
Platform	Dependent	Independent
Components	Limited	Rich set
Look & Feel	OS-based	Customizable

13. Why is Swing preferred over AWT?

- More components
- Better UI
- Platform independent
- Supports MVC architecture

14. Examples of Swing components

- JButton
- JLabel
- JTextField
- JFrame

15. What is an event?

An event is an **action** like:

- Button click
- Key press
- Mouse movement

16. What is event handling?

The process of **responding to events**.

17. What is ActionListener?

An interface used to handle **button click events**.

18. What is actionPerformed()?

A method that is executed when an **action event occurs**.

19. What are event classes?

Classes that represent events.

Examples:

- ActionEvent
- MouseEvent
- KeyEvent

20. What is the delegation event model?

Java uses this model where:

- Event source → generates event
- Listener → handles event

21. What is an event source?

The object that **generates events**

Example: Button

22. What is a listener?

An object that **listens and responds** to events.

23. How do you register a listener?

Using methods like:

```
button.addActionListener(this);
```

25. Why do we use implements ActionListener?

To handle button click events in a class.

26. Why is setLayout(null) used?

To use **absolute positioning** (manual placement).

27. Why use JPasswordField instead of JTextField?

To **hide password characters**.

28. Why convert password to String?

new String(passText.getPassword())

Because getPassword() returns a **char array**.

29. What is JOptionPane used for?

To show **dialog messages** (alerts, confirmations).

30. What does setVisible(true) do?

Makes the window **visible on screen**.

31. What are Adapters?

An **adapter class** is a class that implements a listener interface but gives **default (empty) implementations** of all its methods. This lets you override **only the methods you actually need**.

Listener Interface	Adapter Class
Must implement all methods	Only override required methods
More code	Less code
Example: MouseListener	Example: MouseAdapter

Adapter Class	Used For
MouseAdapter	Mouse events
KeyAdapter	Keyboard events
WindowAdapter	Window events
FocusAdapter	Focus events

Important Listener and methods:

java.awt.event Package

1. ActionListener

void actionPerformed(ActionEvent e)

2. MouseListener

void mouseClicked(MouseEvent e)

void mousePressed(MouseEvent e)

void mouseReleased(MouseEvent e)

void mouseEntered(MouseEvent e)

void mouseExited(MouseEvent e)

3. MouseMotionListener

void mouseDragged(MouseEvent e)

```
void mouseMoved(MouseEvent e)
```

4. KeyListener

```
void keyPressed(KeyEvent e)
```

```
void keyReleased(KeyEvent e)
```

```
void keyTyped(KeyEvent e)
```

5. WindowListener

```
void windowOpened(WindowEvent e)
```

```
void windowClosing(WindowEvent e)
```

```
void windowClosed(WindowEvent e)
```

```
void windowIconified(WindowEvent e)
```

```
void windowDeiconified(WindowEvent e)
```

```
void windowActivated(WindowEvent e)
```

```
void windowDeactivated(WindowEvent e)
```

6. FocusListener

```
void focusGained(FocusEvent e)
```

```
void focusLost(FocusEvent e)
```

7. ItemListener (Checkbox, RadioButton state change)

```
void itemStateChanged(ItemEvent e)
```

8. AdjustmentListener (ScrollBars)

```
void adjustmentValueChanged(AdjustmentEvent e)
```

11. Database programming with java.sql

```
import java.sql.*;
```

```
// Data Access Object (DAO) for Student table
```

```
public class StudentDAO {  
    private Connection conn;
```

```
    // Constructor receives DB connection
```

```
    public StudentDAO(Connection conn) {  
        this.conn = conn;  
    }
```

```
    // Insert a student
```

```
    public void insertStudent(String name, int age) throws SQLException {  
        String sql = "INSERT INTO student (name, age) VALUES (?, ?)";  
        try (PreparedStatement ps = conn.prepareStatement(sql)) {  
            ps.setString(1, name);  
            ps.setInt(2, age);  
            int rows = ps.executeUpdate();  
            System.out.println(rows + " student inserted.");  
        }  
    }
```

```
    // Search student by ID
```

```
    public void searchStudent(int id) throws SQLException {
```

```

String sql = "SELECT * FROM student WHERE id=?";
try (PreparedStatement ps = conn.prepareStatement(sql)) {
    ps.setInt(1, id);
    try (ResultSet rs = ps.executeQuery()) {
        if (rs.next()) {
            System.out.println("ID: " + rs.getInt("id"));
            System.out.println("Name: " + rs.getString("name"));
            System.out.println("Age: " + rs.getInt("age"));
        } else {
            System.out.println("Student not found.");
        }
    }
}
}

```

```

// Fetch all students
public void fetchAllStudents() throws SQLException {
    String sql = "SELECT * FROM student";
    try (Statement stmt = conn.createStatement();
        ResultSet rs = stmt.executeQuery(sql)) {

        System.out.println("\n--- All Students ---");
        while (rs.next()) {
            System.out.println("ID: " + rs.getInt("id") +
                               ", Name: " + rs.getString("name") +
                               ", Age: " + rs.getInt("age"));
        }
    }
}

```

```

// Update student details by ID
public void updateStudent(int id, String name, int age) throws SQLException {
    String sql = "UPDATE student SET name=?, age=? WHERE id=?";
    try (PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setString(1, name);
        ps.setInt(2, age);
        ps.setInt(3, id);
        int rows = ps.executeUpdate();
        System.out.println(rows + " student updated.");
    }
}

```

```

// Delete student by ID
public void deleteStudent(int id) throws SQLException {
    String sql = "DELETE FROM student WHERE id=?";
    try (PreparedStatement ps = conn.prepareStatement(sql)) {

```

```

        ps.setInt(1, id);
        int rows = ps.executeUpdate();
        System.out.println(rows + " student deleted.");
    }
}

import java.sql.*;
import java.util.Properties;
import java.io.FileInputStream;
import java.io.IOException;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {

        Properties props = new Properties();

        // Load DB properties
        try (FileInputStream fis = new FileInputStream("db.properties")) {
            props.load(fis);
        } catch (IOException e) {
            e.printStackTrace();
            return;
        }

        String url = props.getProperty("db.url");
        String user = props.getProperty("db.user");
        String pass = props.getProperty("db.pass");

        // DB connection
        try (Connection conn = DriverManager.getConnection(url, user, pass);
            Scanner sc = new Scanner(System.in)) {

            System.out.println("Connected to database successfully.");

            StudentDAO dao = new StudentDAO(conn); // Create DAO object

            while (true) {
                System.out.println("\n1. Insert Student\n2. Search Student\n3. Fetch All Students\n4.
Update Student\n5. Delete Student\n6. Exit");
                System.out.print("Choose option: ");
                int choice = sc.nextInt();
                sc.nextLine(); // Consume newline

                switch (choice) {

```

```

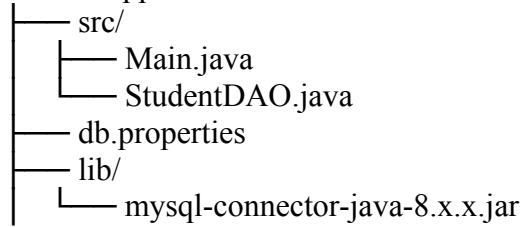
        case 1:
            System.out.print("Enter name: ");
            String name = sc.nextLine();
            System.out.print("Enter age: ");
            int age = sc.nextInt();
            dao.insertStudent(name, age);
            break;
        case 2:
            System.out.print("Enter ID to search: ");
            int id = sc.nextInt();
            dao.searchStudent(id);
            break;
        case 3:
            dao.fetchAllStudents();
            break;
        case 4:
            System.out.print("Enter ID to update: ");
            int updateId = sc.nextInt();
            sc.nextLine();
            System.out.print("Enter new name: ");
            String newName = sc.nextLine();
            System.out.print("Enter new age: ");
            int newAge = sc.nextInt();
            dao.updateStudent(updateId, newName, newAge);
            break;
        case 5:
            System.out.print("Enter ID to delete: ");
            int deleteId = sc.nextInt();
            dao.deleteStudent(deleteId);
            break;
        case 6:
            System.out.println("Exiting...");
            return;
        default:
            System.out.println("Invalid option!");
    }
}

} catch (SQLException e) {
    e.printStackTrace();
}
}
}

```


Project Structure:

StudentApp/



db.properties:

db.url=jdbc:mysql://localhost:3306/school

db.user=root

db.pass=password